# Introduction to



python

## with Application to Bioinformatics

- Day 3

# Day 3

- **Session 1: Functions and f-strings**
    - Review Day 2
    - Build a greeting function; practice arguments, return values, and formatted output
    - exercise 1
- **Session 2: Command-line arguments**
    - Inspect `sys.argv`
    - Make the greeting script accept a name
    - exercise 2
- **Session 3: Parse a data file**
    - Inspect the IMDb file format
    - Parse each line into a dictionary, then build a parser function
- **Session 4: Search and filter**
    - Search parsed movie records for the highest rating
    - Use sets for unique genres and membership questions
    - Revisit the Day 2 IMDb task
    - Exercise 3
- **Project time and quizzes**

# Quiz: Review Day 2

Go to Canvas, `Modules -> Day 3 -> Review Day 2`

~20 minutes

# Tuples (Q 1&2)

**1. Which of the following variables are of the type tuple?**

```
a = (1, 2, 3, 4)
a = ([1, 2], 'a', 'b')
```

**2. What is the difference between a tuple and a list?**

A tuple is immutable while a list is mutable

In [3]:

```
myTuple    = (1, 2, 3)
myList     = [1, 2 ,3]
```

**2. What is the difference between a tuple and a list?**

A tuple is immutable while a list is mutable

In [3]:
```
myTuple    = (1, 2, 3)
myList     = [1, 2 ,3]
```

In [4]:
```
myList[2]  = 4
myList
```

Out[4]:
```
[1, 2, 4]
```

**2. What is the difference between a tuple and a list?**
A tuple is immutable while a list is mutable

In [3]:
```
myTuple    = (1, 2, 3)
myList     = [1, 2 ,3]
```

In [4]:
```
myList[2]  = 4
myList
```

Out[4]:
```
[1, 2, 4]
```

In [5]:
```
try:
    myTuple[2] = 4
except Exception as e:
    print(e)
print(f"tuple contents: {myTuple}")
```

```
'tuple' object does not support item assignment
tuple contents: (1, 2, 3)
```

Is it true that we can never modify the content of a tuple?

Is it true that we can never modify the content of a tuple?

In [6]:
```
myTuple = (1, 2, [1,2,3])
print(myTuple)
myTuple[2][2] = 4
print(myTuple)
```

```
(1, 2, [1, 2, 3])
(1, 2, [1, 2, 4])
```

## Is it true that we can never modify the content of a tuple?

In [6]:
```
myTuple = (1, 2, [1,2,3])
print(myTuple)
myTuple[2][2] = 4
print(myTuple)
```

```
(1, 2, [1, 2, 3])
(1, 2, [1, 2, 4])
```

- The immutability of tuples in Python means that **the structure of the tuple itself cannot be changed**, you cannot add, remove, or replace elements in the tuple.
- However, if a tuple contains mutable objects like lists, dictionaries, or other objects, the contents of those mutable objects can still be changed.

# How to structure the code (Q 3)

**3. What does pseudocode mean?**
Writing down the steps you intend to include in your code in more general language

# Things to Consider When Writing Pseudocode

- Decide on the desired output.
- Identify the input files you have.
- Examine the structure of the input – can it be iterated over?
- Determine where the necessary information is located.
- Assess if you need to store information while iterating:
    - Use lists for ordered data.
    - Use sets for unique, non-duplicate entries.
    - Use dictionaries for structured, key-value information.
- After collecting the required data, decide how to process it.
- Determine if you'll need to write your results to a file.

**Writing pseudocode before actual coding is a good habit.**

# Functions and methods (Q 4&5)

**4. What are the following examples of?**

```
len([1, 2, 3, 4])
print("my text")
```

Functions

# Functions and methods (Q 4&5)

**4. What are the following examples of?**

```
len([1, 2, 3, 4])
print("my text")
```

Functions

**5. What are the following examples of?**

```
"my\ttext".split("\t")
[1, 2, 3].pop()
```

Methods

## What are the differences between a `function` and a `method`?

| **Function** | **Method** | |---|---| | Standalone block of code | Function associated with an object | | Called independently, e.g. `functionName()` | Called on an instance of a class, e.g. `obj.methodName()` | | Not tied to any object or class | Tied to the objects they are called on | | Defined outside of a class | Defined within a class | |

**6. Calculate the average of the list `[1,2,3.5,5,6.2]` to one decimal, using Python**

In [7]:
```
myList = [1, 2, 3.5, 5 ,6.2]
round(sum(myList)/len(myList),1)
```

Out[7]:
```
3.5
```

**7. Take the list `['I','know','Python']` as input and output the string 'I KNOW PYTHON'**

In [8]:
```
my_list   = ['I','know','Python']

" ".join(my_list).upper()
```

Out[8]:
```
'I KNOW PYTHON'
```

# Functions and f-strings

## Code with me Session 1: Make a greeter

Write a function that accepts a name and returns a greeting string. Then call it for two different names.

Pull up relevant documentation:
For reference:
functions: https://www.w3schools.com/python/python_functions.asp
f-strings: https://www.w3schools.com/python/python_string_formatting.asp

Write with me in ipython:

```
pixi run ipython
```

# Functions and f-strings

## Code with me Session 1: Make a greeter

Write a function that accepts a name and returns a greeting string. Then call it for two different names.

Pull up relevant documentation:
For reference:
functions: https://www.w3schools.com/python/python_functions.asp
f-strings: https://www.w3schools.com/python/python_string_formatting.asp

Write with me in ipython:

```
pixi run ipython
```

In [9]:
```
def hello(name):
    return f"Greetings, {name}!"

print(hello("Mattis"))
print(hello("Kalle Karlsson"))
```

```
Greetings, Mattis!
Greetings, Kalle Karlsson!
```

Do exercise 1 ~ 10 minutes

## Using functions to break up complexity

- Rule of thumb: more than 3 levels of indentation --> break into functions

When doing this:

```
def parse_file(file_path):
    with open(file_path) as fh:
        for line in fh:
            for word in line.split():
                # do something
```

Try this:

```
def parse_words(line):
    words = []
    for word in line.split():
        words.append(word.lower())
    return words

def parse_lines(fh):
    parsed_lines = []
    for line in fh:
        parsed_lines.append(parse_words(line))
    return parsed_lines

def parse_file(file_path):
    with open(file_path) as fh:
        return parse_lines(fh)
```

# Session 2: Make the greeter reusable from the command line ~20 minutes

**Problem:** run the same greeting program for different people without editing the source each time.

- break into pairs
- see if you can figure out how to use argv based on the script behavior and content
- then create a script according to instructions below (also found in exercise 2)
- its OK to ask for help if you get stuck!

First inspect `downloads/print_argv.py` and predict what each command will pass to the program:

```
pixi run python downloads/print_argv.py
pixi run python downloads/print_argv.py Mattis
pixi run python downloads/print_argv.py Kalle Karlsson
```

In a new file called `greet.py`, copy your `hello` function from Session 1, then use `sys.argv` to collect the name. Run it from the workspace root with `python greet.py Mattis`.

example solution:

```
from sys import argv

def hello(name):
    return f"Greetings, {name}!"

name = f'{argv[1]}'
print(hello(name))
```

# Session 3: Turn file lines into movie records

**Problem:** make the IMDb file usable by later code. Parsing is one task; asking questions about parsed movies is a different task.

The file is pipe-delimited. Each data row has these fields:

```
votes | rating | year | runtime_seconds | image_url | genres | title
```

Rows beginning with `#` are comments. Open the file, inspect the header and one data row, and identify the fields we need for later questions: title, year, rating, runtime, and genres.

# Session 3: Turn file lines into movie records

**Problem:** make the IMDb file usable by later code. Parsing is one task; asking questions about parsed movies is a different task.

The file is pipe-delimited. Each data row has these fields:

`votes | rating | year | runtime_seconds | image_url | genres | title`

Rows beginning with `#` are comments. Open the file, inspect the header and one data row, and identify the fields we need for later questions: title, year, rating, runtime, and genres.

In [10]:
```
with open(IMDB_FILE_PATH, encoding="utf-8") as movie_file:
    for line in movie_file:
        if not line.startswith("#"):
            print(line.rstrip())
            break
```

```
    126807|   8.5|1957|5280|https://images-na.ssl-images-amazon.
com/images/M/MV5BOTI5Nzc0OTMtYzBkMS00NjkxLThmM2UtNjM2ODgxN2M5NjN
kXkEyXkFqcGdeQXVyNjQ2MjQ5NzM@._V1_.jpg|Drama,War|Paths of Glory
```

## Step 1: Parse one row

Before writing a file loop, solve the smaller problem: turn one pipe-delimited row into a dictionary. Use named keys so later code does not need to remember numeric field positions. More information about dictionaries: https://www.w3schools.com/python/python_dictionaries.asp

## Step 1: Parse one row

Before writing a file loop, solve the smaller problem: turn one pipe-delimited row into a dictionary. Use named keys so later code does not need to remember numeric field positions. More information about dictionaries: https://www.w3schools.com/python/python_dictionaries.asp

In [11]:
```python
def parse_movie_line(line):
    fields = line.rstrip("\n").split("|")
    genres = []
    for genre in fields[5].split(","):
        genres.append(genre.strip().lower())

    return {
        "title": fields[6].strip(),
        "year": int(fields[2].strip()),
        "rating": float(fields[1].strip()),
        "runtime_seconds": int(fields[3].strip()),
        "genres": genres,
    }
```

Check this parser with one real data row. The result is a dictionary with a numeric rating and year, a runtime in seconds, and a list of lowercase genre names.

Check this parser with one real data row. The result is a dictionary with a numeric rating and year, a runtime in seconds, and a list of lowercase genre names.

In [12]:
```
with open(IMDB_FILE_PATH, encoding="utf-8") as movie_file:
    for line in movie_file:
        if not line.startswith("#"):
            first_movie = parse_movie_line(line)
            break

first_movie
```

Out[12]:
```
{'title': 'Paths of Glory',
 'year': 1957,
 'rating': 8.5,
 'runtime_seconds': 5280,
 'genres': ['drama', 'war']}
```

# Step 2: Open the file

## Step 2: Open the file

In [13]:
```
def parse_movie_lines(movie_file):
    movies = []
    for line in movie_file:
        if line.startswith("#"):
            continue
        movies.append(parse_movie_line(line))
    return movies

def load_movies(path):
    with open(path, encoding="utf-8") as movie_file:
        return parse_movie_lines(movie_file)

movies = load_movies(IMDB_FILE_PATH)
len(movies), movies[0], movies[-1]
```

Out[13]:
```
(250,
 {'title': 'Paths of Glory',
  'year': 1957,
  'rating': 8.5,
  'runtime_seconds': 5280,
  'genres': ['drama', 'war']},
 {'title': 'The Shining',
  'year': 1980,
  'rating': 8.4,
  'runtime_seconds': 8520,
  'genres': ['drama', 'horror']})
```

# Session 4: Search and filter the parsed records

The parser has one job: turn file rows into records. Now use those records to answer questions. Start with a tiny list you can reason about by eye; include a tie so the search has to keep more than one answer.

## Problem A: Find every movie tied for the highest rating

Implement `find_highest_rated(movie_records)` so it returns a list of movie dictionaries.

## Problem A: Find every movie tied for the highest rating

Implement `find_highest_rated(movie_records)` so it returns a list of movie dictionaries.

In [14]:
```
toy_movies = [
    {"title": "A", "rating": 8.0},
    {"title": "B", "rating": 9.1},
    {"title": "C", "rating": 9.1},
    {"title": "D", "rating": 7.5},
]

def find_highest_rated(movie_records):
    highest_rating = movie_records[0]["rating"]
    highest_movies = [movie_records[0]]
    for movie in movie_records[1:]:
        if movie["rating"] > highest_rating:
            highest_rating = movie["rating"]
            highest_movies = [movie]
        elif movie["rating"] == highest_rating:
            highest_movies.append(movie)
    return highest_movies

find_highest_rated(toy_movies)
```

Out[14]:
```
[{'title': 'B', 'rating': 9.1}, {'title': 'C', 'rating': 9.1}]
```

The toy case preserves both movies rated 9.1. Now apply the same search function to the 250 parsed records. The function can be tested independently of file reading because it accepts a list of records.

The toy case preserves both movies rated 9.1. Now apply the same search function to the 250 parsed records. The function can be tested independently of file reading because it accepts a list of records.

In [15]:
```
for movie in find_highest_rated(movies):
    print(f"{movie['title']} ({movie['rating']})")
```

```
The Shawshank Redemption (9.3)
```

# Problem B: How many distinct genres are in the dataset?

A movie can have multiple genres, and many movies share genres. Collect each genre in a set, then inspect the count and sorted values. The set is useful here because it can only contain unique items. https://www.w3schools.com/python/python_sets.asp

## Problem B: How many distinct genres are in the dataset?

A movie can have multiple genres, and many movies share genres. Collect each genre in a set, then inspect the count and sorted values. The set is useful here because it can only contain unique items. https://www.w3schools.com/python/python_sets.asp

In [16]:
```
unique_genres = set()
for movie in movies:
    for genre in movie["genres"]:
        unique_genres.add(genre)

len(unique_genres), sorted(unique_genres)
```

Out[16]:

```
(22,
 ['action',
  'adventure',
  'animation',
  'biography',
  'comedy',
  'crime',
  'drama',
  'family',
  'fantasy',
  'film-noir',
  'historical',
  'history',
  'horror',
  'music',
  'musical',
  'mystery',
  'romance',
  'sci-fi',
  'sport',
  'thriller',
  'war',
  'western'])
```

## Problem C: Filter records by a genre

Use membership testing to keep only movies whose genre list contains `thriller`.

## Problem C: Filter records by a genre

Use membership testing to keep only movies whose genre list contains `thriller`.

In [17]:
```
def filter_by_genre(movie_records, requested_genre):
    matching_movies = []
    requested_genre = requested_genre.strip().lower()
    for movie in movie_records:
        if requested_genre in movie["genres"]:
            matching_movies.append(movie)
    return matching_movies
# to optimize membership checking we can convert the list of genres to s
for movie in movies:
    movie["genres"] = set(movie["genres"])
thriller_movies = filter_by_genre(movies, "Thriller")
print(f"Found {len(thriller_movies)} thriller movies")
for movie in thriller_movies[:5]:
    print(movie["title"])
```

```
Found 65 thriller movies
North by Northwest
On the Waterfront
Aliens
Léon: The Professional
Inception
```

# Summary

- Use `sys.argv` to deal with arguments passed to the python script from the command line
    - `sys.argv[0]` is the program name
    - `sys.argv[1]` is is the first argument and so on
- `f-strings` formatting is a convenient and recommended way to format the string
    - Extra reading about string formatting: https://www.w3schools.com/python/python_string_formatting.asp

## Day 3, Exercise 3 (~30 min)

- Restructure and write the output to a new file
- Canvas -> Modules -> Day 3 -> IMDb exercise - day 3 -> 4
- Work in pairs

PyQuiz 3.2

---

## Project time

In [ ]: